

Deep learning:

Date _____
Page _____

Deep Learning

ML VS Deep learning

ML	DL
→ It's a subset of AI	It's a subset of ML
→ Data less req'd	Huge data req'd.
→ Speed is faster	Speed is slower
→ used in: Spam detection, recommendation system	used in image recognition, self-driving cars.
→ Ex: SVM, Decision Tree	for ex: CNN, RNN, ANN

• Deep learning:

Deep learning is a branch of AI and a subset of ML that uses multilayered neural networks to automatically learn patterns from data.

Neural Network = Input → Learn patterns → Output pred.

"Layers in ANN" (Artificial Neural Network)

→ Input layer → Hidden layer → Output layer

* Simple Neural Network (fully connected)

Ex: 3-4-2 Network (3 inputs, 1 hidden layer with 4 neurons, 2 outputs)

The diagram illustrates a fully connected neural network with three layers:

- Input layer (3 neurons):** Labeled x_1, x_2, x_3 . Inputs are labeled Inp 1, Inp 2, and Inp 3.
- Hidden layer (4 neurons):** Labeled h_1, h_2, h_3, h_4 .
- Output layer (2 neurons):** Labeled y_1, y_2 . Outputs are labeled Output-1 and Output-2.
- Bias nodes:** Labeled b_1 and b_2 at the bottom, with arrows pointing to the hidden and output layers respectively.

Connections are shown between all neurons in adjacent layers.

Parameter count Breakdown:

Component	calculation	Parameters
1. Wt: input $\rightarrow$ Hidden	input $\times$ hidden $= 3 \times 4$	12
2. Biases: Hidden Layer (1 bias for each neuron)	hidden = 4	4
3. Wt: Hidden $\rightarrow$ output (connection from 4 hidden neurons to 2 output)	hidden $\times$ output	8
4. Biases: Output layer (1 bias for output neuron)	noutput = 2	2
Total Parameters		26

Summary:

• Network Architecture: 2-4-2

• Total parameters: 26

• formula:

$$\text{Total} = (n_{in} \times n_{hidden}) + n_{hidden} + (n_{hidden} \times n_{out}) + n_{out}$$

(weight + biases)
(wt + biases)

where $n_{in} = 3$, $n_{hidden} = 4$, $n_{out} = 2$

$$\text{Total} = (3 \times 4) + 4 + (4 \times 2) + 2 = 26$$

How parameters are calculated:

- wt. betn two layers: # connection = #neurons in previous layer $\times$ #neurons in next layer
- Biases in a layer: biases = neurons in the layer
- Total parameters = All wt + All biases

Key Takeaway: parameters are the learnable values that the neural network adjusts during training to make better prediction.

• Parameters Count of Real-World Models:

① ~~GPT-40~~

Model	organization	TYPE	Parameters
① Gpt-40	open AI	LLM	~1.7 Trillion
② GPT-4 Turbo	open AI	LLM	~1.0 Trillion

* Why More Parameters = more capacity to learn complex patterns.

Parameters are the learnable values in a neural network
More parameter $\rightarrow$ more flexibility to fit many diff relationship in the data.

$\rightarrow$ But More parameters do not always mean better performance. It also depends on data quality, amount of data, architecture, and training Model.

SMALL MODEL (fewer Parameters)

In small model (9 parameters)

- can learn simple patterns (like a st. line)
- less flexible boundary
- May underfit complex data.

Large model (41 parameters)

- can learn more complex patterns (curved boundary)
- can fit more complicated data.

- ① feed data : Input data and true label are fed into the model.

Ex:

Input Feature (x)			True Label (y)
0.9	0.7	-0.1	1
-1.5	0.3	0.8	0
0.6	0.2	0.9	1

Batch of data

- ② forward pass:

Data passes through the network layer by layer to produce prediction

- ③ compute loss:

Compare $\hat{y}$ compare prediction ($\hat{y}$) with true label (y) using a loss function.

Ex: Binary Cross-Entropy Loss:

Gradients show how much each wt contributed to loss.

Example of Backward Pass:

- finds ~~set~~ which neuron caused error
- Adjusts its wt. to reduce error next time

"Summary Flow"

Feed Data $\rightarrow$ Forward Pass $\rightarrow$ Compute Loss $\rightarrow$ Backward pass $\rightarrow$ update weights $\rightarrow$ Repeat.

Input features	Weights	Multiplication weighted input	Summation + Bias
x_1	w_1	$x_1 \cdot w_1$	$z = x_1 w_1 + x_2 w_2$
x_2	w_2	$x_2 \cdot w_2$	$+ b$
$\rightarrow$ Summation + Bias $z = x_1 w_1 + x_2 w_2$ — Activation function (Non-linear)			

update weight:

update wt each weight in the opposite direction of the gradient. (to reduce loss)

Gradient Descent Update Rule:

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial L}{\partial w}$$

where: η = learning rate

$\frac{\partial L}{\partial w}$ = gradient of loss wrt weight.

Activation Functions :-

Ex :-

1. Data set :-

	x_1	x_2	y
1	1	2	1
2	0	-1	0
3	2	0.5	1
4	-1	1.0	0

 $y=1 \rightarrow$ positive class $y=0 \rightarrow$ negative class

2. FORWARD PASS :-

	x_1	x_2	$z = w_1x_1 + w_2x_2 + b$	$y = \sigma(z) = \frac{1}{1+e^{-z}}$	Predicted class
1.	1.0	2.0	$(0.5)(1.0) + (-1)(2) + 0.2$ $= -1.3$	0.2142	0
2.	0	-1	$(0.5)(0) + (-1)(-1) + 0.2$ $= 1.2$	0.7685	1
3.	2	0.5	$(0.5)(2) + (-1)(0.5) + 0.2$ $= 0.7$	0.6682	1
4.	-1	1.0	$(0.5)(-1.0) + (-1)(1.0) + 0.2$ $= -1.3$	0.2142	0

3. To find Error / Loss calculation $[L = -[y_i \log(\hat{y}_i) + (1-y_i) \log(1-\hat{y}_i)]]$

y	$\hat{y}$	L_i calculation	L_i
1.	1	$-[1 \cdot \log(0.2142) + 0 \cdot \log(1-0.2142)]$ $= 1.5404$	1.5404
2.	0	$-[0 \cdot \log(0.7685) + 1 \cdot \log(1-0.7685)]$ $= 1.4644$	1.4644
3.	1	$-[1 \cdot \log(0.6682) + 0 \cdot \log(1-0.6682)]$ $= 0.4037$	0.4037
4.	0	$-[0 \cdot \log(0.2142) + 1 \cdot \log(1-0.2142)]$ $= 0.2418$	0.2418

Total loss (sum)

$$\sum_{i=1}^4 l_i = 1.5404 + 1.4644 + 0.4037 + 0.2418 \\ = 3.6503$$

Average loss (Mean)

$$J = \frac{1}{m} \sum_{i=1}^m l_i = \frac{3.6503}{4} = 0.9126$$

* PERCEPTION VS Logistic Regression:

Perception

Logistic Regression

- | | |
|---------------------------------|--------------------------------|
| (i) Output is either 0 or 1 | output is 0 to 1 (probability) |
| (ii) Boundary is st. line | can be curved |
| (iii) It is used in Basic Prob. | used in Real problem |
| (iv) less complexity | Slightly more complex |
| (v) less stable | More stable & reliable |
| (vi) uses step function | uses sigmoid function |

- Both are supervised learning alg
- Both uses for binary classification
- Both create a linear decision boundary

PERCEPTRON VS MULTI-LAYER PERCEPTRON:

Perceptron (Single-layer)

Multi-layer Perceptron (MLP)

① Simple neural network with one layer

Neural Network with multiple layers (inp + hid + out)

② layers: only input → output

layers: Input → hidden layers → output

③ Step function

uses sigmoid, Relu, Tanh, etc

④ Decision boundary: st. line

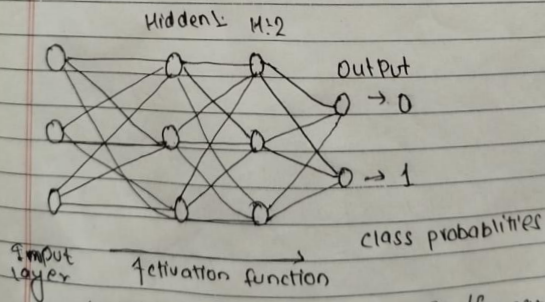
Decision boundary: complex / non-linear boundaries

ility)

⑤ Rare Real-World use

widely used in deep learning

* Multi-layer Perceptron (MLP)



Multi-layer perceptron is a feedforward artificial neural network with one or more hidden layers that uses non-linear activation functions and backpropagation to learn complex learn complex patterns from data.

PERCEPTRON VS MULTI-LAYER PERCEPTRON:

Perceptron (Single-layer)

Multilayer Perceptron (MLP)

① Simple neural network with one layer

Neural Network with multiple layers (inp + hid + out)

② layers: only input → output

layers: Input → hidden layers → output

③ Step function

uses sigmoid, ReLU, Tanh, etc

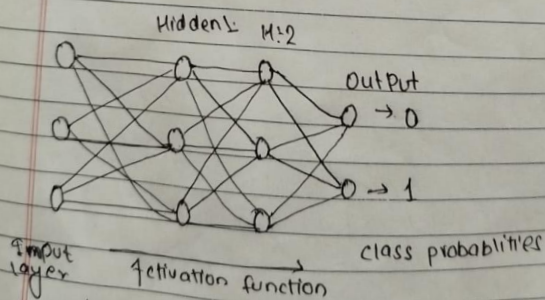
④ Decision boundary: st. line

Decision boundary: complex / non-linear boundaries

⑤ Rare Real-World use

widely used in deep learning

* Multi-layer Perceptron (MLP)



Multilayer perceptron is a feedforward artificial neural network with one or more hidden layers that uses non-linear activation functions and backpropagation to learn complex patterns from data.

compute loss $\rightarrow$ compute gradients $\rightarrow$ update parameters
 (using y & $\hat{y}$) $\frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial w_2}, \frac{\partial L}{\partial b}$ (w_1, w_2, w_3)

$\rightarrow$ Repeat for all data
 for each epochs until
 loss is minimized.

<1> Full Process of Multilayered perceptron

① Input Data (x, y)

- x (input features) : eg. age, BP
- y (Labels/output) : actual answer (0 or 1)

② Neural Network (MLP)

contains input layers, then hidden layer and then output layer. Each layer has weights and Bias.

$$\text{Formula } Z = XW + b$$

$$A = f(Z)$$

③ Prediction ($\hat{y}$) - forward pass

Data follows from input $\rightarrow$ output

Model gives prediction $\hat{y}$ (y hat)

Ex: output = $[0.7, 0.2, 0.1]$

④ Compare & Loss:

Compare ~~prediction~~ Predicted ($\hat{y}$) vs ~~actual~~ actual (y).

calculate loss: MSE, Cross Entropy

⑤ update wt (Back propagation)

$$\text{compute gradients} = \frac{\partial L}{\partial w}, \frac{\partial L}{\partial b}$$

update weights:

$$w = w - \eta \frac{\partial L}{\partial w}$$

$$b = b - \eta \frac{\partial L}{\partial b}$$

η = learning rate

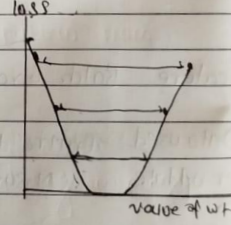
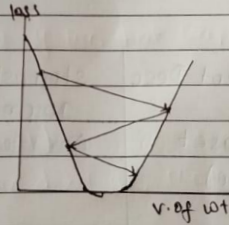
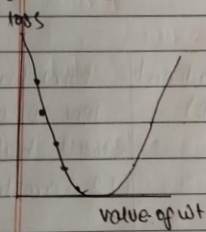
* This process repeats for many epochs.
Goal $\rightarrow$ minimize loss

Feature	Batch Gradient Desc	Stochastic Gradient Descent (SGD)	Mini-batch gradient Descent
Data used	Entire dataset	One sample	Small batch
Per update	(N sample)		(11 samples)
Update per epoch	1	N	N/B
Speed	slow	fast (very freq. update)	fast
Stability	very stable	Noisy/unstable	good balance
Memory	High	Low	medium

Learning rate concept:

- Learning rate (η) = Step Size used in gradient descent
It controls how much wt. are updated

feature	Good learning rate	Large learning	Ver. large learning
step size	small and balanced	Big	Ver. big
Movement	Smooth descent	Zig-zag	Jump randomly
stability	stable	unstable	Highly unstable
Accuracy	High	Medium	Ver. low



Overfitting: Training vs Validation Loss

Epoch	Training loss	Validation loss
0	1.80	1.75
5	1.25	1.10
10	0.85	0.80
15	0.60	0.65
20	0.45	0.55
25	0.35	0.58
30	0.28	0.68
35	0.22	0.85
40	0.18	1.05
45	0.16	1.25
50	0.14	1.50

Key Takeaway:

- Initially, both training & validation loss decrease
- After a point, validation loss starts increasing while training loss keeps decreasing
- This gap indicates overfitting
- The best model is usually at the minimum validation loss

ML - Key Terms Exp:

- ① Dataset :
Collection of data used to train and evaluate the model. Ex: Imaging of cats and dogs.
- ② Feature : Age, Salary, ... (input attributes/variable given to the model)
- ③ Label / Target :
Correct ans that the model should predict
Ex: Dog, House price.
- ④ Model :
Mathematical system that learns pattern from data.
- ⑤ Parameter (wt & bias) :
Value learned by the Model during training
- ⑥ Prediction :
The output produced by the model. ($\hat{y}$)
- ⑦ Loss / Cost (J) :
Measure how wrong the pred. is compared to the actual label. Lower loss = better performance
- ⑧ Gradient ΔJ :
The slope of the loss function w.r.t a parameter

⑨ Learning Rate:

controls the step size during each update.

Small $\alpha \rightarrow$ slow learning

Large $\alpha \rightarrow$ fast "

v. large $\alpha \rightarrow$ may diverge

Ex: $\alpha = 0.01$

⑩ Gradient Descent:

Alg. used to minimize the loss by updating parameters in the direction of neg. gradient

$$w_{new} = w_{old} - \alpha \cdot \frac{dI}{dw}$$
 Repeat until convergence

⑪ Epoch:

one complete pass through the entire training dataset

If dataset has 1000 samples and ~~concept~~ batch size = 100, then 1 epoch = 10 iterations.

⑫ Iteration:

Each batch $\rightarrow$ one step.

⑬ Convergence:

when the loss stop changing significantly or changes very little after many epochs.

⑭ Hyperparameters:

learning rate (α), Batch size, No. of epochs, model architecture, optimizer, regularization strength

These are not learned by the model we set them before training.